15.4.1 Software Engineering Methods

If you expect to build high-quality software, you must understand the problem to be solved. You must also be capable of creating a design that conforms to the problem while at the same time exhibiting characteristics that lead to software that exhibits the quality dimensions and factors discussed in Section 15.2.

In Part Two of this book, we presented a wide array of concepts and methods that can lead to a reasonably complete understanding of the problem and a comprehensive design that establishes a solid foundation for the construction activity. If you apply those concepts and adopt appropriate analysis and design methods, the likelihood of creating high-quality software will increase substantially.

15.4.2 Project Management Techniques

The impact of poor management decisions on software quality has been discussed in Section 15.3.6. The implications are clear: If (1) a project manager uses estimation to verify that delivery dates are achievable, (2) schedule dependencies are understood and the team resists the temptation to use shortcuts, (3) risk planning is conducted so problems do not breed chaos, software quality will be affected in a positive way.

In addition, the project plan should include explicit techniques for quality and change management. Techniques that lead to good project management practices are discussed in Part Four of this book.

15.4.3 Machine Learning and Defect Prediction

Defect prediction [Mun17] is an important part of identifying software components that may have quality concerns. Defect prediction models use statistical techniques to examine the relationships among combinations of software metrics and software components containing known software defects. They can be an efficient and effective way for software developers to quickly identify defect-prone classes. This can reduce costs and development times [Mal16].

Machine learning is an application of artificial intelligence (AI) techniques that provide systems with the ability to learn and improve from experience without being explicitly programmed. Stated another way, machine learning focuses on the development of computer programs that can access data and use the data to learn for themselves. Machine learning techniques can be used to automate the process of discovering predictive relationships between software metrics and defective components [Ort17], [Li16], [Mal16].

Machine learning systems process large data sets containing representative combinations of metrics for defective and nondefective software components. These data are used to tune classification algorithms. Once the system has built a prediction model though this type of training, it can be used for quality assessment and defect prediction on data associated with future software products. Building these types of classifiers is a big part of what modern data scientists do. More discussion on the use of data science and software engineering appears in Appendix 2 of this book.

15.4.4 Quality Control

Quality control encompasses a set of software engineering actions that help to ensure that each work product meets its quality goals. Models are reviewed to ensure that

they are complete and consistent. Code may be inspected to uncover and correct errors before testing commences. A series of testing steps is applied to uncover errors in processing logic, data manipulation, and interface communication. A combination of measurement and feedback allows a software team to tune the process when any of these work products fails to meet quality goals. Quality control activities are discussed in detail throughout the remainder of Part Three of this book.

15.4.5 Quality Assurance

Quality assurance establishes the infrastructure that supports solid software engineering methods, rational project management, and quality control actions—all pivotal if you intend to build high-quality software. In addition, quality assurance consists of a set of auditing and reporting functions that assess the effectiveness and completeness of quality control actions. The goal of quality assurance is to provide management and technical staff with the data necessary to be informed about product quality, thereby gaining insight and confidence that actions to achieve product quality are working. Of course, if the data provided through quality assurance identifies problems, it is management's responsibility to address the problems and apply the necessary resources to resolve quality issues. Software quality assurance is discussed in detail in Chapter 17.

15.5 SUMMARY

Concern for the quality of the software-based systems has grown as software becomes integrated into every aspect of our daily lives. But it is difficult to develop a comprehensive description of software quality. In this chapter, quality has been defined as an effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.

A wide variety of software quality dimensions and factors has been proposed over the years. All try to define a set of characteristics that, if achieved, will lead to high software quality. McCall's and the ISO 25010 quality factors establish characteristics such as reliability, usability, maintainability, functionality, and portability as indicators that quality exists.

Every software organization is faced with the software quality dilemma. In essence, everyone wants to build high-quality systems, but the time and effort required to produce "perfect" software are simply unavailable in a market-driven world. The question becomes, Should we build software that is "good enough"? Although many companies do just that, there is a significant downside that must be considered.

Regardless of the approach that is chosen, quality does have a cost that can be discussed in terms of prevention, appraisal, and failure. Prevention costs include all software engineering actions that are designed to prevent defects in the first place. Appraisal costs are associated with those actions that assess software work products to determine their quality. Failure costs encompass the internal price of failure and the external effects that poor quality precipitates.

Software quality is achieved through the application of software engineering methods, solid management practices, and comprehensive quality control—all supported by a software quality assurance infrastructure. In the chapters that follow, quality control and assurance are discussed in some detail.

PROBLEMS AND POINTS TO PONDER

15.1. Describe how you would assess the quality of a university before applying to it. What factors would be important? Which would be critical?

15.2. Using the definition of software quality proposed in Section 15.2, do you think it's possible to create a useful product that provides measurable value without using an effective process? Explain your answer.

15.3. Using the subattributes noted for the ISO 25010 quality factor "maintainability" in Section 15.2.1, develop a set of questions that explore whether or not these attributes are present. Follow the example shown in Section 15.2.2.

15.4. Describe the software quality dilemma in your own words.

15.5. What is "good enough" software? Name a specific company and specific products that you believe were developed using the good enough philosophy.

15.6. Considering each of the four aspects of the cost of quality, which do you think is the most expensive and why?

15.7. Do a Web search and find three other examples of "risks" to the public that can be directly traced to poor software quality. Consider beginning your search at http://catless.ncl.ac.uk/risks.

15.8. Are *quality* and *security* the same thing? Explain.

15.9. Explain why it is that many of us continue to live by Meskimen's law. What is it about the software business that causes this?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

16

REVIEWS—A RECOMMENDED APPROACH

Software reviews are a "filter" for the software process work flow. Too few, and the flow is "dirty." Too many, and the flow slows to a trickle. Reviews are applied at various points during software engineering and serve to uncover errors and defects. Software reviews "purify" software engineering work products, including requirements and design models, code, and testing data. Using metrics, you can determine which reviews work and emphasize them and at the same time remove ineffective reviews from the flow to accelerate the process.

KEY CONCEPTS

bugs 326
cost effectiveness 329
defect amplification 327
defects 326
error density 328
errors 326
informal reviews 331
record keeping 333
review reporting 333
technical reviews 326

QUICK LOOK

What is it? You'll make mistakes as you develop software engineering work products. There's no shame in that—as long as you try hard, very hard, to find and correct the mistakes before they are delivered to end users. Technical reviews are the most effective mechanism for finding mistakes early in the software process.

Who does it? Software engineers perform technical reviews, also called peer reviews, with their colleagues. As we discussed in Chapters 3 and 4, sometimes it is wise to include other stakeholders in these reviews.

Why is it important? If you find an error early in the process, it is less expensive to correct. In addition, errors have a way of amplifying as the process proceeds. So a relatively minor error left untreated early in the process can be amplified into a major set of errors later in the project. Finally, reviews save time by reducing the amount of rework that will be required late in the project.

What are the steps? Your approach to reviews will vary depending on the type of review you select. In general, six steps are employed, although not all are used for every type of review: planning, preparation, structuring the meeting, noting errors, making corrections (done outside the review), and verifying that corrections have been performed properly.

What is the work product? The output of a review is a list of issues and/or errors that have been uncovered. In addition, the technical status of the work product is also indicated.

How do I ensure that I've done it right? First, select the type of review that is appropriate for your development culture. Follow the guidelines that lead to successful reviews. If the reviews that you conduct lead to higher-quality software, you've done it right.

Many different types of reviews can be conducted as part of software engineering. Each has its place. An informal meeting around the coffee machine is a form of review, if technical problems are discussed. A formal presentation of software architecture to an audience of customers, management, and technical staff is also a form of review. In this book, however, we focus on *technical* or *peer reviews,* exemplified by *casual reviews, walkthroughs,* and *inspections.* A technical review (TR) is the most effective filter from a quality control standpoint. Conducted by software engineers and other stakeholders for all project team members, the TR is an effective means for uncovering errors and improving software quality.

16.1 Cost Impact of Software Defects

Within the context of the software process, the terms *defect* and *fault* are synonymous. Both imply a quality problem that is discovered *after* the software has been released to end users (or to another framework activity in the software process). In earlier chapters, we used the term *error* to depict a quality problem that is discovered by software engineers (or others) *before* the software is released to the end user (or to another framework activity in the software process).

INFO



Bugs, Errors, and Defects

The goal of software quality control, and in a broader sense, quality management in general, is to remove quality problems in the software. These problems are referred to by various names—*bugs, faults, errors,* or *defects,* to name a few. Are these terms synonymous, or are there subtle differences between them?

In this book we make a clear distinction between an *error* (a quality problem found *before* the software is released to other stakeholders or end users) and a *defect* (a quality problem found only *after* the software has been released to end users or other stakeholders).[1] We make this distinction because errors and defects have very different economic, business, psychological, and human impacts. As software engineers, we want to find and correct as many errors as possible before the customer and/or end user encounter them. We want to avoid defects—because defects (justifiably) make software people look bad.

It is important to note, however, that the temporal distinction made between errors and defects in this book is *not* mainstream thinking. The general consensus within the software engineering community is that defects and errors, faults, and bugs are synonymous. That is, the point in time that the problem was encountered has no bearing on the term used to describe the problem. Part of the argument in favor of this view is that it is sometimes difficult to make a clear distinction between pre- and postrelease (e.g., consider an incremental process used in agile development).

Regardless of how you choose to interpret these terms, recognize that the point in time at which a problem is discovered does matter and that software engineers should try hard—*very* hard—to find problems before their customers and end users encounter them.

1 If software process improvement is considered, a quality problem that is propagated from one process framework activity (e.g., **modeling**) to another (e.g., **construction**) can also be called a "defect" because the problem should have been found before a work product (e.g., a design model) was "released" to the next activity.

The primary objective of a formal technical review (FTR) is to find errors before they are passed on to another software engineering activity or released to the end user. The obvious benefit of technical reviews is the early discovery of errors so that they do not propagate to the next step in the software process.

A number of industry studies indicate that design activities introduce between 50 and 65 percent of all errors (and ultimately, all defects) during the software process. However, review techniques have been shown to be up to 75 percent effective [Jon86] in uncovering design flaws. By detecting and removing a large percentage of these errors, the review process substantially reduces the cost of subsequent activities in the software process. We have known this for decades, and yet there are still many developers who do not believe the time spent on reviews is almost always less than the time required to rewrite bad code [Yad17].

16.2 Defect Amplification and Removal

Defect amplification is a concept originally proposed almost four decades ago [IBM81]. It helps to justify the effort expended on software reviews. In essence, defect amplification makes the following argument—an error introduced early in the software engineering work flow (e.g., during requirement modeling) and undetected, can and often will be amplified into multiple errors during design. If those errors are not uncovered (using effective reviews), they themselves may be further amplified into still more errors during coding. A single error introduced early and not uncovered and corrected can amplify into multiple errors later in the process. *Defect propagation* is a term used to describe the impact an undiscovered error has on future development activities or product behavior [Vit17].

As a development team moves deeper into the software process, the cost of finding and fixing an error grows. This simple reality is exacerbated by defect amplification and propagation because a single error may become multiple errors downstream. The cost of finding and fixing a single error can be signficant, but the cost to find and fix multiple errors propagated by a single earlier error is substantially more significant.

To conduct reviews, you must expend time and effort, and your development organization must spend money. However, the reality of defect amplification and propagation leaves little doubt that you can pay now or pay much more later. This is what *technical debt* (Chapter 11) is all about [Xia16] [Vit17].

16.3 Review Metrics and Their Use

Technical reviews are one of many actions that are required as part of good software engineering practice. Each action requires dedicated human effort. Because available project effort is finite, it is important for a software engineering organization to understand the effectiveness of each action by defining a set of metrics (Chapter 23) that can be used to assess their efficacy.